

Symbiosis Institute of Technology, Pune

Faculty of Engineering

CSE- Academic Year 2025-26

Compiler Construction Lab Batch 2022-26

Lab Assignment No: - 2	
Name of Student	Pranav Suri
PRN No.	22070122152
Batch	2022-26
Class	CSE – B3
Academic Year & Semester	4 th year, 7 th Semester
Date of Performance	
Title of Assignment	Count the number of comments, keywords, identifiers, words, lines, and spaces from the input file.
Practice Questions	1. Lex program counts the number of comments, keywords, identifiers, words, lines, and spaces from the input file. 2. Lex Program to print the total characters, white spaces, tabs in the given input file. 3. Lex code to count the total number of tokens.
Source Code	1. analyzer.l %{ #include <stdio.h> #include <string.h>

```

int comment_count = 0, keyword_count = 0,
identifier_count = 0; int word_count = 0, space_count = 0,
line_count = 0;

char *keywords[] = {
    "int", "float", "char", "return", "if", "else", "while", "for", "void",
    "main" };

int is_keyword(char *word) {
    for (int i = 0; i < sizeof(keywords)/sizeof(keywords[0]); i++) {
        if (strcmp(word, keywords[i]) == 0)
            return 1;
    }
    return 0;
}

%}

%option noyywrap

%%

"/".* { comment_count++; }
"/"([^\]\+[\^/])\+ "/" { comment_count++; }
[a-zA-Z_][a-zA-Z0-9_]* {
    word_count++;
    if (is_keyword(yytext)) keyword_count++;
    else identifier_count++;
}
[ \t]+ { space_count += yyleng; }
\n { line_count++; }
. { word_count++; } // symbols and others
%%

int main() {
    yylex();
    printf("Total Lines : %d\n", line_count);
    printf("Total Spaces : %d\n", space_count);
    printf("Total Words : %d\n", word_count);
    printf("Total Keywords : %d\n", keyword_count);
    printf("Total Identifiers: %d\n", identifier_count);
    printf("Total Comments : %d\n", comment_count);
    return 0;
}

input.txt
//single line comment

/*

```

	multi line comment
--	--------------------

```
*/
```

```
int main() {  
    int x = 10;  
    float y = 3.14;  
    char c = 'B';  
    if (x>5) {  
        x=x+1;  
    }  
}
```

2. a2.l

```
%{  
#include <stdio.h>  
int char_count = 0, space_count = 0, tab_count = 0;  
%}  
%%  
[ \\t\\n] {  
    char_count++;  
    if (yytext[0] == ' ') space_count++;  
    else if (yytext[0] == '\\t') tab_count++;  
}  
.\\n { char_count++; }  
%%  
int yywrap() { return 1; }  
int main() {  
    yylex();  
    printf("Total Characters: %d\\n", char_count);  
    printf("Total Spaces : %d\\n", space_count);  
    printf("Total Tabs : %d\\n", tab_count);  
    return 0;  
}
```

input.txt

```
int main() {  
    int x = 10;  
    float y = 3.14;  
    char c = 'B';  
    return 0;  
}
```

3. token.l

```
%{  
#include <stdio.h>
```

	<pre>int token_count = 0; {%}</pre>
--	-------------------------------------

	<pre>%% [\t\n]+ ; [0-9]+ { token_count++; } "int" "float" "if" "else" "for" "while" "return" "char" "void" "main" { token_count++; } [a-zA-Z_][a-zA-Z0-9_]* { token_count++; } "==" "!=" "<=" ">=" "<" ">" "=" { token_count++; } [+\-*/%{}()<,>] { token_count++; } . ; %% int yywrap() { return 1; } int main() { yylex(); printf("Total Tokens: %d\n", token_count); return 0; } input.txt int main() { int x = 10; int y = 0; if(y<x) { y = y+1; } return 0; }</pre>
Output Screen shot	1.

```

student@ubuntu:~$ flex analyzer.l
flex: could not create lex.yy.c
student@ubuntu:~$ ^[[200~rm lex.yy.c
rm: command not found
student@ubuntu:~$ rm lex.yy.c
rm: remove write-protected regular file 'lex.yy.c'? y
student@ubuntu:~$ flex analyzer.l
student@ubuntu:~$ gcc lex.yy.c -o analyzer -lfl
student@ubuntu:~$ ./analyzer < input.txt
ash: input.txt: No such file or directory
student@ubuntu:~$ ls
ah          calc.l      Demo1.sh   Documents  input      Public     y.ta
analyzer    calc.tab.c  Demo2.sh   Downloads  lex.yy.c   snap       y.ta
analyzer.l  calc.tab.h  Demo3.sh   expr.l     Music      Templates
alc         calc.y      Desktop    expr.y     Pictures   Videos
student@ubuntu:~$ ./analyzer < input.txt
total Lines : 12
total Spaces : 20
total Words : 41
total Keywords : 6
total Identifiers: 7
total Comments : 2
student@ubuntu:~$ █

```

2.

```

student@ubuntu:~$ flex a2.l
student@ubuntu:~$ gcc lex.yy.c -o a2 -lfl
student@ubuntu:~$ ./a2 < input.txt
Total Characters: 71
Total Spaces : 12
Total Tabs : 4

```

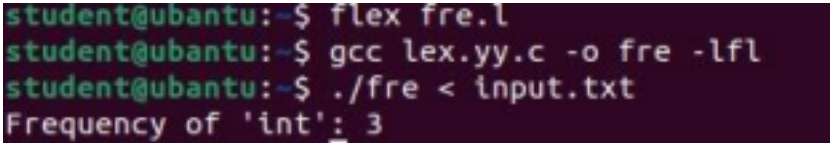
3.

```

student@ubuntu:~$ flex token.l
student@ubuntu:~$ gcc lex.yy.c -o token -lfl
student@ubuntu:~$ ./token < input.txt
total Tokens: 33

```

Post lab questions	1. Lex program to count the frequency of the given word in a file. CODE: <pre>%{ #include <stdio.h> #include <string.h> int freq = 0; char target_word[] = "int"; // You can change this %}</pre>
--------------------	--

	<pre> %% [a-zA-Z_][a-zA-Z0-9_]* { if (strcmp(yytext, target_word) == 0) freq++; } . \\n ; // Ignore everything else %% int yywrap() { return 1; } int main() { yylex(); printf("Frequency of '%s': %d\\n", target_word, freq); return 0; } INPUT: int main() { int x = 10; int y = 0; if(y<x) { y = y+1; } return 0; } OUTPUT: </pre>  <pre> student@ubuntu:~\$ flex fre.l student@ubuntu:~\$ gcc lex.yy.c -o fre -lfl student@ubuntu:~\$./fre < input.txt Frequency of 'int': 3 </pre>
Conclu sio n	We have successfully counted the number of comments, keywords, identifiers, words, lines, and spaces from the input file.

